

for something you have yet to even learn. And if concurrency is that important you might want to take a look at Go, a language developed by Google and is part of some of their processes. Here are a few usage scenarios if you are still stubborn on using D:

1. Creating a project from scratch with a faster interface creation time period.
2. If you need fast low-level programming binaries while making a program from scratch.
3. Native support for concurrency.

Major Design Goals of D

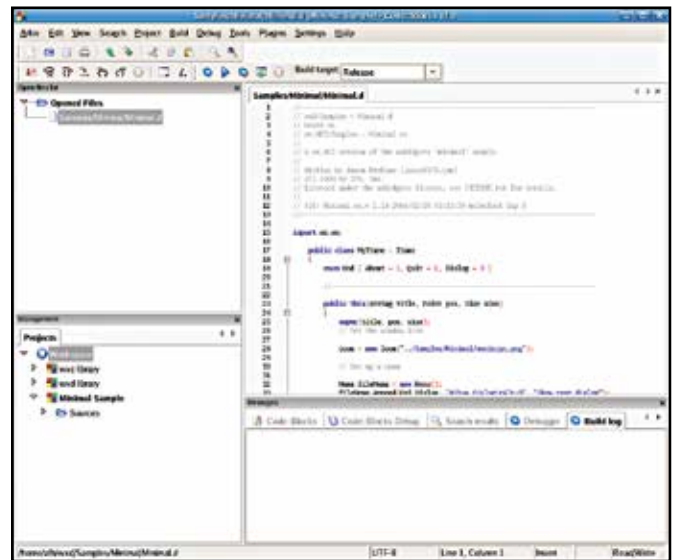
Every language has certain design goals that are the guidelines for the development of the language and here are some of the design principles for D.

1. Make writing code easier that is portable from compiler to compiler, machine to machine and from operating system to operating system.
2. To provide syntactic and semantic constructs to eliminate common programming mistakes.
3. Support memory safe programming.
4. Have a shorter learning curve for programmers comfortable with C or C++ or Java.
5. Be compatible with the binary interface of the local C application
6. Have context free grammar
7. Incorporate contract programming and unit testing
8. Help build lightweight and standalone programs
9. Reduce documentation creation costs

Who should use D?

Programmers who regularly use lint or similar code analysis tools before the code is even compiled. Those who keep warning levels at maximum with warning categorised as errors. For projects that consist of million lines of code that need built-in testing and verification. Then there are numerical programmers, since D supports features like floating point precision, core support for floating types and defined behaviours for NaN's and infinities.

D's compiler structure is modular. The lexical analyser and parser are totally independent of each other and the semantic analyser so if



D programming with Code::Blocks as the IDE

you were to write a simple tool to manipulate the source code then you can modify parts of the compiler rather than the entire compiler. This also means the source code can be broken down as tokens for specialised applications. And lastly, for those who believe that C++ is just not enough a little modern touch is necessary for their project. The language as a whole is kept as close as possible to C++ which means a transition into D shouldn't be difficult at all.

Who shouldn't use D?

Let's face it, C++ has matured over the ages so there is no way someone is going to sit down and convert his/her million line C/C++ program into D. D is similar to C/C++ but it doesn't compile C/C++ source code out of the box. However, D supports legacy C API's quite well.

People getting into programming shouldn't choose D over the more favoured legacy languages like C/Java/Basic.

Getting started with D

There are three popular compilers for the D programming language.

*coding matters



Learn Swift for free

>>Apple has released a free PDF on iTunes for Swift, the new programming language that will make its debut with iOS 8 later this year and is to make coding for the new iOS much easier. Get it here.

<http://dgit.in/FreeSwift>



Top languages to learn

>>IEEE Spectrum has curated a list of the most popular programming languages based on job demand, popularity among different communities and platforms.

<http://dgit.in/TopLanguageList>



Amazon introduces Cloud Watch

>>The new service from Amazon lets you monitor and troubleshoot systems and applications from Amazon EC2 instances. Learn more here:

<http://dgit.in/CloudWatchAmazon>